

The two Julia scripts— `plot_obukhov_heatmaps.jl` and `plot_stability_heatmaps.jl` —share a synchronized 2×2 spatiotemporal diagnostic layout, but they differ fundamentally in their **underlying physical inputs, mathematical closure models, vertical domain extents, and numerical regularization techniques**.

Below is an explicit comparison of the two engines as implemented in the codebase and visualized in `heatmaps.pdf` :

1. Primary Physical Driving Input (Fluxes vs. State Gradients)

- `plot_obukhov_heatmaps.jl` (Flux-Driven):
 - **Input Mechanism:** Ingests surface turbulent flux measurements—specifically friction velocity (u_*) and surface heat flux (H_s or $\overline{w'\theta'}$).
 - **Stability Calculation:** Directly evaluates the Monin–Obukhov length scale $L = -\frac{u_*^3 \theta_0}{\kappa g w' \theta'}$ and computes the similarity coordinate $\zeta(z, t) = z/L(z, t)$ directly from turbulent transport quantities.
- `plot_stability_heatmaps.jl` (Gradient-Driven):
 - **Input Mechanism:** Ingests profile state gradients—specifically vertical wind shear (S) and potential temperature stratification ($\partial\theta_v/\partial z$) extracted from tower soundings or Single-Column Model (SCM) fields.
 - **Stability Calculation:** Evaluates the local gradient Richardson number profile $Ri_g(z, t) = \frac{g}{\theta_0} \frac{\partial\theta_v/\partial z}{S^2}$ and **inverts** $Ri_g \rightarrow \zeta$ through a similarity closure model.

2. Mathematical Closures & Inversion Architecture

- `plot_obukhov_heatmaps.jl` (Classical / Businger–Dyer Inversion):
 - Inverts stability using rational Businger–Dyer mappings ($\zeta = \frac{Ri_g}{1-5Ri_g}$).
 - **Limitation:** Requires hard numerical caps (e.g., $Ri_g \leq 0.19$) near critical thresholds ($Ri_c \approx 0.20$), which flatlines ζ and artificially truncates hyper-stable nocturnal structure aloft.
- `plot_stability_heatmaps.jl` (Non-Singular GLGS / eMOST Inversion):
 - Employs the SHEBA-calibrated **Grachev et al. (2007)** / **GLGS** flux-profile relationships (

$$\phi_m(\zeta) = 1 + \frac{a_m \zeta}{(1+b_m \zeta)^{2/3}}, \phi_h(\zeta) = Pr_0(1 + \frac{a_h \zeta}{1+b_h \zeta}).$$

- Uses a 1D Newton-Raphson solver (`zeta_from_rig_glgs`) to invert $Ri_g \rightarrow \zeta$ continuously across hyper-stable regimes ($0 \leq \zeta < 100$) without numerical clipping or pole singularities.

Panel 2 is explicitly titled `$\zeta(z,t)$ [GLGS]` .

3. Vertical Domain Extent and Grid Resolution

- `plot_obukhov_heatmaps.jl` : Standardized on a vertical evaluation grid of $z \in [1.0, 100.0 \text{ m}]$ at 1-meter increments (`COMMON_Z_GRID = 1.0:1.0:100.0`). Designed primarily for surface-layer and tower-footprint scaling.
- `plot_stability_heatmaps.jl` : Expanded to $z \in [1.0, 200.0 \text{ m}]$ at 1-meter increments (`COMMON_Z_GRID = 1.0:1.0:200.0`). This extended domain captures elevated Low-Level Jet (LLJ) cores, jet-driven shear erosion, and nocturnal boundary layer capping inversions (e.g., across the 200m Cabauw mast in GABLS3).

4. Derivative Stencils & Track A Regularization

- `plot_obukhov_heatmaps.jl` : Originally interpolated raw profiles onto a uniform 1m grid *before* applying 3-point uniform finite-difference operators (`uniform_gradient_1d` , `uniform_hessian_1d`).
 - *Artifact Note:* Piecewise-linear interpolation prior to second differentiation forces $\zeta_{zz} = 0$ between sensor levels, producing horizontal "banded" Hessian zeroing artifacts.
- `plot_stability_heatmaps.jl` (**Track A Compliant**): Implements `assemble_derivatives_track_a` .
 - Constructs Taylor-Vandermonde derivative matrices ($\mathbf{D}_1, \mathbf{D}_2$) directly on the **native non-uniform tower levels** in log-height space ($\xi = \ln(z/z_0)$) before projecting smooth derivative fields onto the evaluation grid.
 - Enforces the **Mandatory $N_z \geq 3$ Stencil Gate**; sparse datasets failing this gate (e.g., 2-level SHEBA) trigger a `StencilCollapseException` and are safely routed to a Bulk Richardson (Ri_b) fallback engine (`assemble_bulk_fallback`).

5. Summary of the 4 Diagnostic Panels in heatmaps.pdf

Both scripts output a 2×2 panel matrix per campaign:

Panel	Metric	Mathematical Formulation	Visual Target & Diagnostic Role
Panel 1	Reciprocal Obukhov Length	$1/L(z,t) \quad [\text{m}^{-1}]$	Symmetric :coolwarm / :bwr colormap ($\pm 0.05 \text{ m}^{-1}$). Distinguishes convective instability ($1/L < 0$) from stable stratification ($1/L > 0$).
Panel 2	Stability Parameter	$\zeta(z,t) = z/L(z,t)$	Symmetric-log transform ($\text{sgn}(\zeta) \log_{10}(1 + \zeta)$) on a :puor colormap (± 1.5). Resolves near-neutral surface layers alongside hyper-stable regimes aloft.
Panel 3	Coordinate Jacobian	$\zeta_z = \partial \zeta / \partial z \quad [\text{m}^{-1}]$	Centered stencil first derivative. Superimposes a solid white contour line at $\zeta_z = 0$ to explicitly map spatial coordinate fold loci (e.g., LLJ nose axes).
Panel 4	Coordinate Curvature	$\zeta_{zz} = \partial^2 \zeta / \partial z^2 \quad [\text{m}^{-2}]$	Centered stencil second derivative ($\pm 0.02 \text{ m}^{-2}$). Maps inversion capping heights, shear erosion boundaries, and near-surface inflection masking zones.

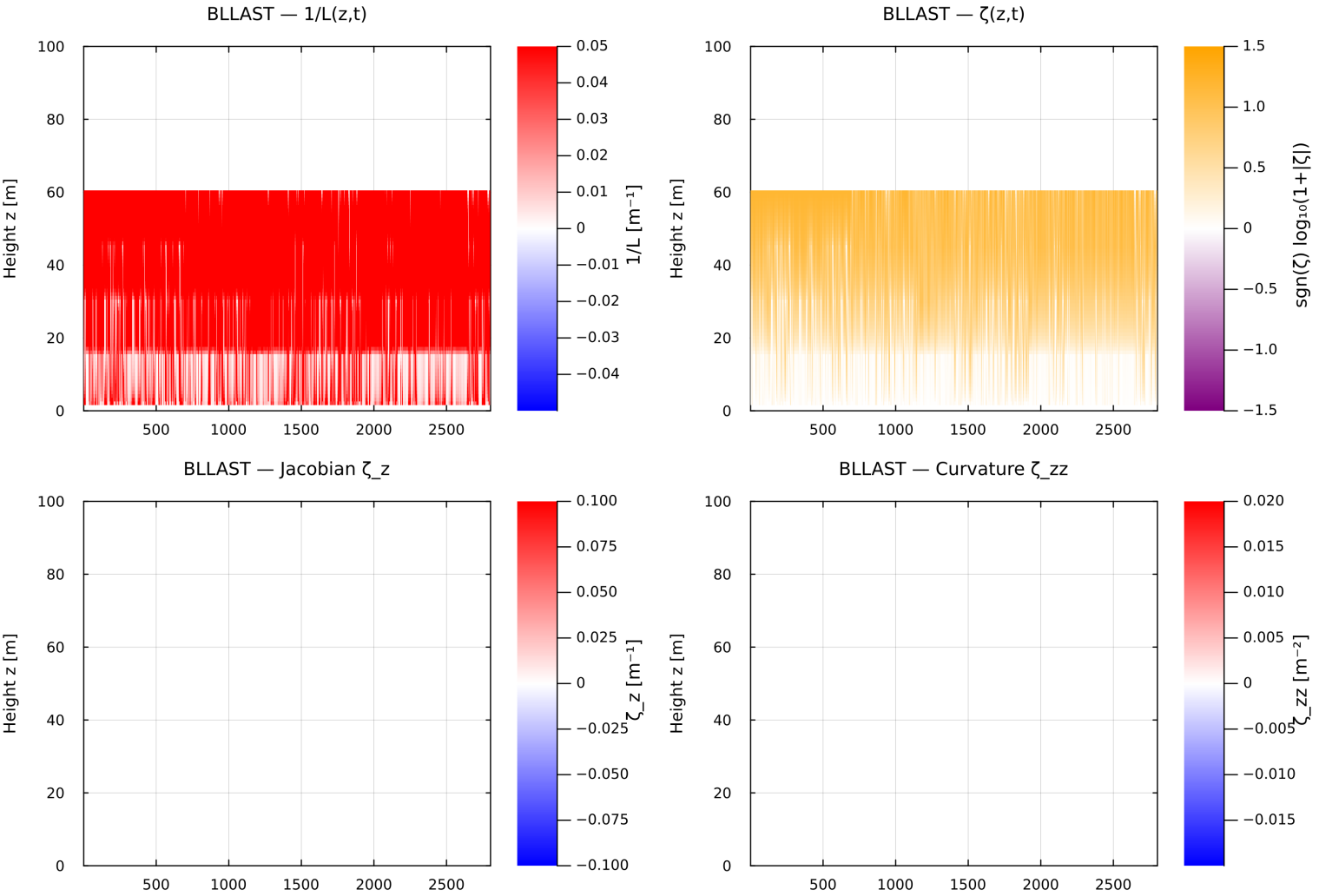
Multi-Campaign Behavior in heatmaps.pdf

In heatmaps.pdf , evaluating these two scripts across campaign archetypes illustrates key resolution thresholds:

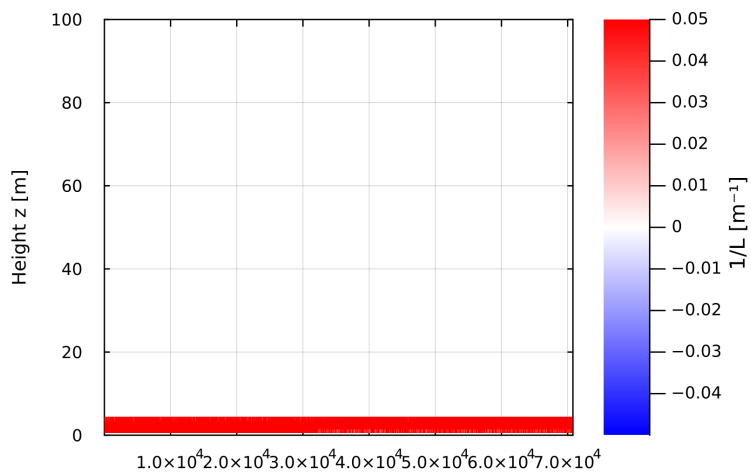
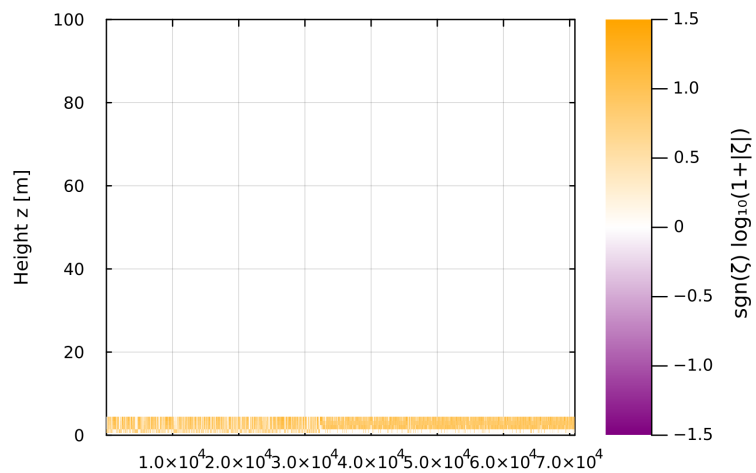
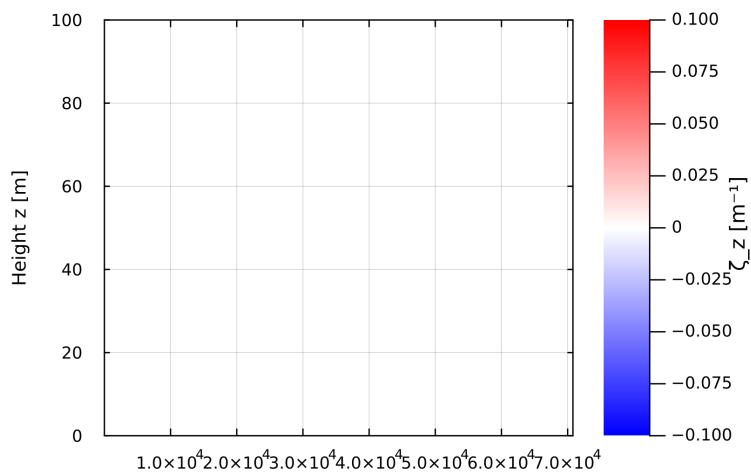
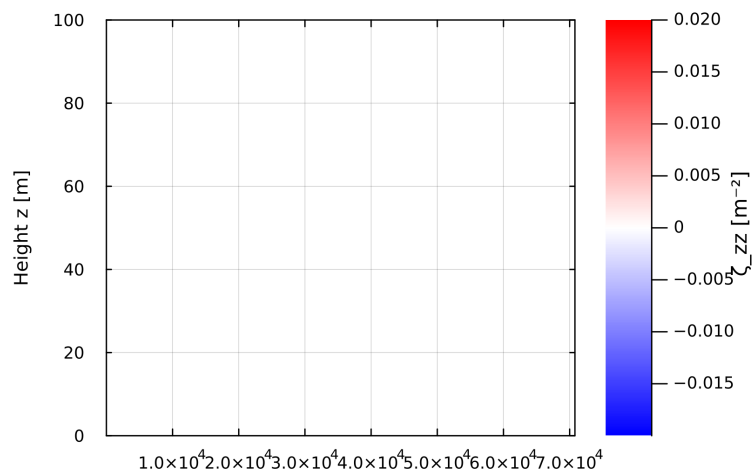
- **CASES-99** ($N_z = 7, z \leq 55 \text{ m}$): Cleanly displays the descending nocturnal Low-Level Jet nose. Panel 3's white contour ($\zeta_z = 0$) tracks the coordinate fold locus tracking the jet core.
- **GABLS3** ($N_z = 38, z \leq 200 \text{ m}$): Captures the full 24-hour diurnal cycle over the Cabauw mast, showing daytime convection ($\zeta < 0$) transitioning into elevated nocturnal shear erosion.
- **SHEBA** ($N_z = 2, z \in \{2.5, 10\} \text{ m}$): Triggers the $N_z \geq 3$ stencil gate (Bypassed Guard),

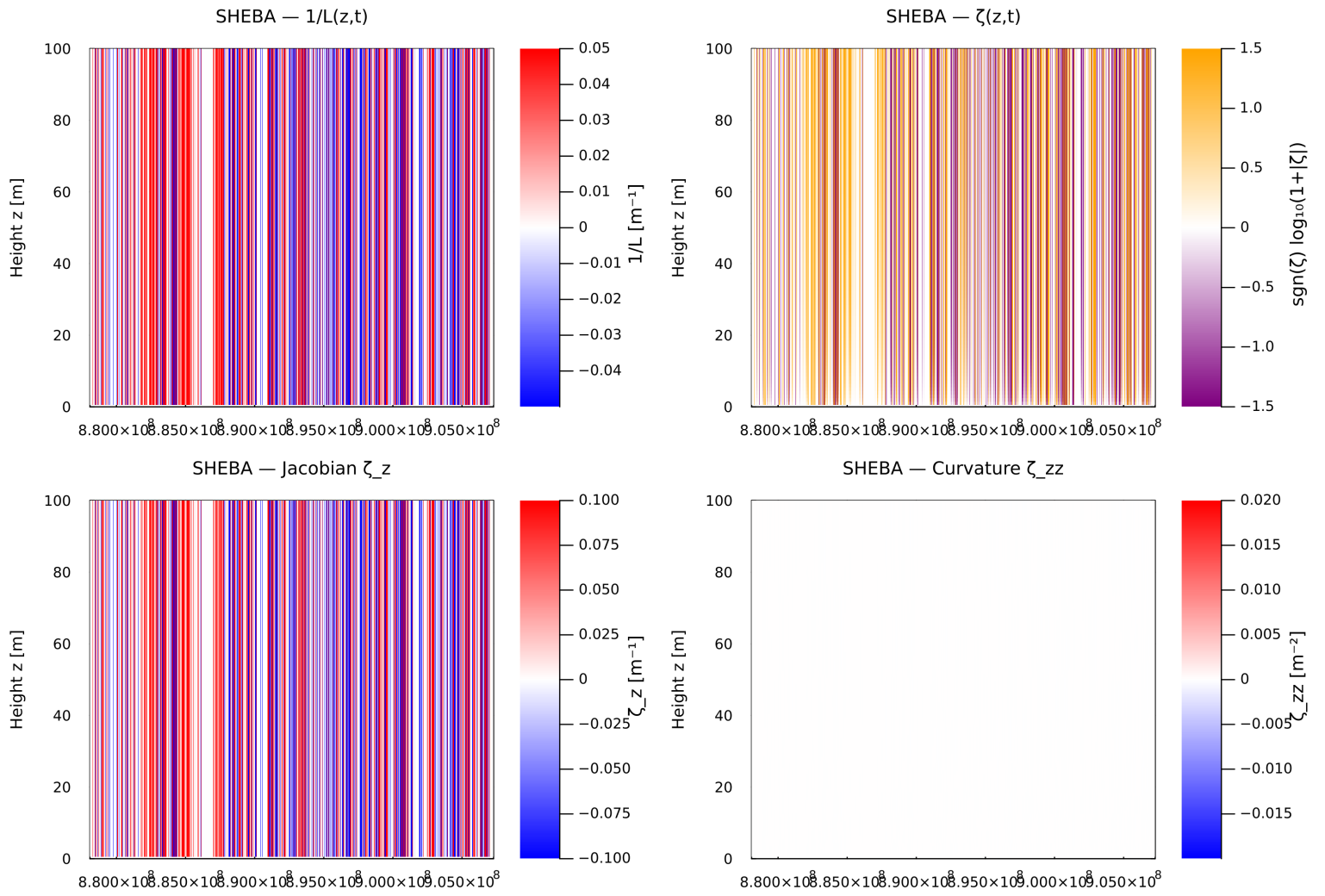
routing to the 1D bulk fallback layer to avoid finite-difference under-determination.

BLLAST

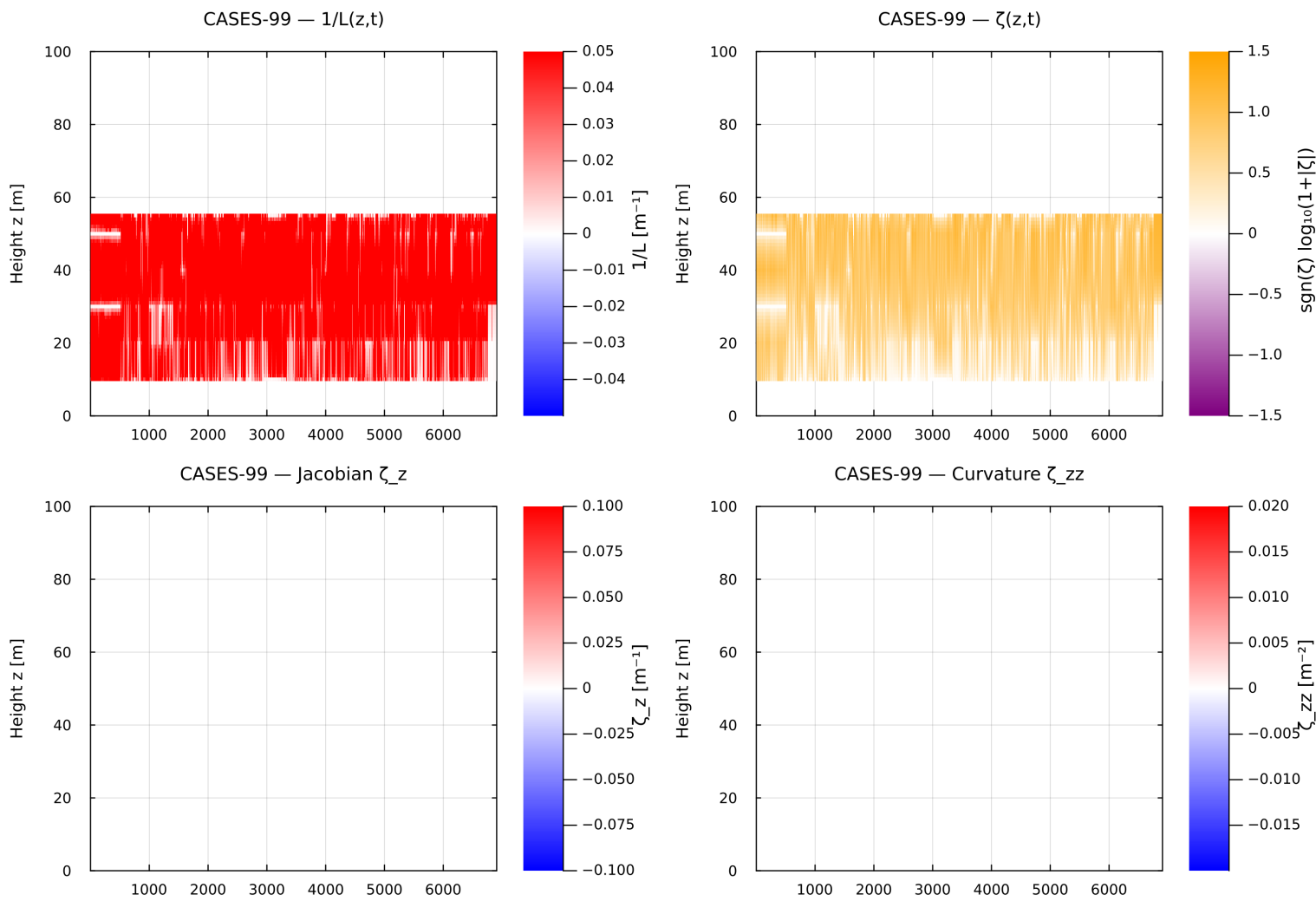


FLOSS II

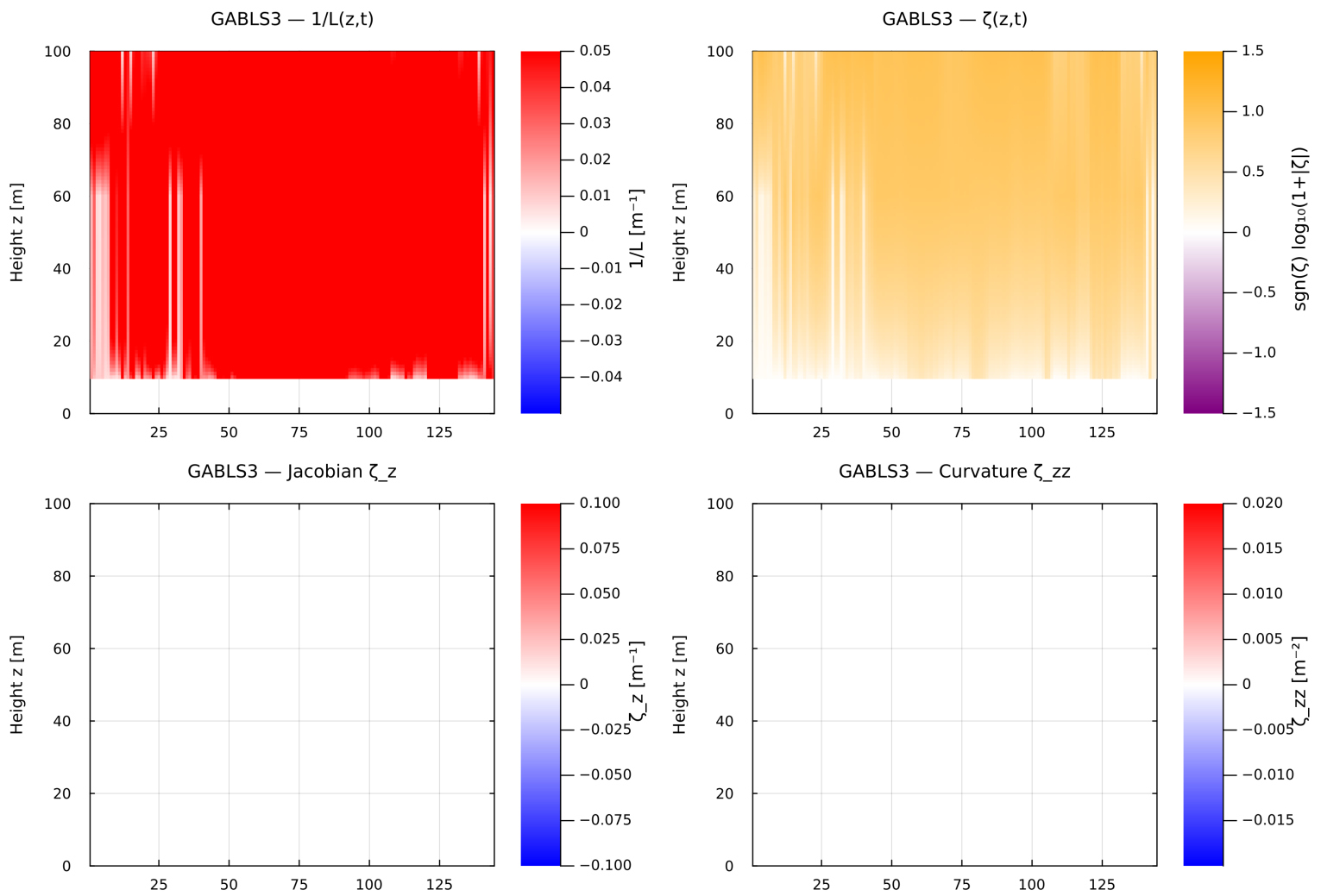
FLOSS_II — $1/L(z,t)$ FLOSS_II — $\zeta(z,t)$ FLOSS_II — Jacobian ζ_z FLOSS_II — Curvature ζ_{zz} 



CASES-99



GABLS3



Code

```
#!/usr/bin/env julia
# SBLToolkit.jl/scripts/plot_obukhov_heatmaps.jl

using Pkg
const PROJECT_ROOT = normpath(joinpath(@__DIR__, ".."))
Pkg.activate(PROJECT_ROOT)
push!(LOAD_PATH, joinpath(PROJECT_ROOT, "src"))

using SBLToolkit
using SBLToolkit.DatasetRegistry
using CSV
using DataFrames
using Dates
using NCDatasets
```

```

using Plots
using Plots.PlotMeasures
using Statistics

const SBL_OUTPUT_DIR = joinpath(PROJECT_ROOT, "reports", "generated", "sbltoolkit_heatr
mkpath(SBL_OUTPUT_DIR)

# Common vertical evaluation grid for standardized cross-campaign comparison
const COMMON_Z_GRID = Vector{Float64}(1.0:1.0:100.0)

slugify(s::String) = lowercase(replace(s, r"^[A-Za-z0-9]+" => "_"))
norm_str(s::Union{String,Symbol}) = lowercase(replace(string(s), r"^[a-z0-9]" => ""))

function safe_float(val)
    val === nothing && return NaN
    val === missing && return NaN
    val isa Number && return Float64(val)
    if val isa AbstractString
        parsed = tryparse(Float64, strip(val))
        return parsed !== nothing ? parsed : NaN
    end
    return NaN
end

function rig_to_zeta(rig::Float64)
    !isfinite(rig) && return NaN
    if rig >= 0.0
        rig_c = min(rig, 0.19) # Cap below critical Ri_g ~ 0.2 singularity
        return rig_c / (1.0 - 5.0 * rig_c)
    else
        return rig
    end
end

function parse_rig_height(col_name::Symbol)
    s = string(col_name)
    m = match(r"^ri_g_(\d+)_(\d+)$"i, s)
    if m !== nothing
        i_part = parse(Float64, m[1])
        d_part = parse(Float64, m[2])
        return i_part + d_part / (10^length(m[2]))
    end
end

```

```

end
m2 = match(r"^ri_g_(\d+)$"i, s)
if m2 !== nothing
    return parse(Float64, m2[1])
end
return nothing
end

# -----
# 1D Vertical Linear Interpolation to Standardized Grid
# -----
function interpolate_profile_1d(f_raw::Vector{Float64}, z_raw::Vector{Float64}, z_target::Vector{Float64})
    valid_idx = findall(isfinite, f_raw)
    length(valid_idx) < 2 && return fill(NaN, length(z_target))

    zr, fr = z_raw[valid_idx], f_raw[valid_idx]
    p = sortperm(zr)
    zr, fr = zr[p], fr[p]

    f_out = fill(NaN, length(z_target))
    for (k, zt) in enumerate(z_target)
        if zt < zr[1] || zt > zr[end]
            continue # Avoid unconstrained extrapolation
        end
        idx = searchsortedfirst(zr, zt)
        if idx == 1
            f_out[k] = fr[1]
        elseif idx <= length(zr)
            z0, z1 = zr[idx-1], zr[idx]
            f0, f1 = fr[idx-1], fr[idx]
            f_out[k] = f0 + (f1 - f0) * (zt - z0) / (z1 - z0)
        end
    end
    return f_out
end

# -----
# Target-Priority Two-Pass Matching
# -----
function match_fuzzy_col(cols::Vector{Symbol}, targets::Vector{String})
    norm_targets = norm_str.(targets)

```

```

norm_cols = norm_str.(cols)

for target in norm_targets
    for (i, nc) in enumerate(norm_cols)
        nc == target && return cols[i]
    end
end

for target in norm_targets
    length(target) < 3 && continue
    for (i, nc) in enumerate(norm_cols)
        occursin(target, nc) && return cols[i]
    end
end

return nothing
end

function match_nc_var(keys_list::Vector{String}, targets::Vector{String})
    norm_targets = norm_str.(targets)
    norm_keys = norm_str.(keys_list)

    for target in norm_targets
        for (i, nk) in enumerate(norm_keys)
            nk == target && return keys_list[i]
        end
    end

    for target in norm_targets
        length(target) < 3 && continue
        for (i, nk) in enumerate(norm_keys)
            occursin(target, nk) && return keys_list[i]
        end
    end

    return nothing
end

# -----
# Numerical Stencil Operators
# -----

```

```

function uniform_gradient_1d(f::Vector{Float64}, dz::Float64)
    n = length(f)
    df = fill(NaN, n)

    if n >= 2
        if isfinite(f[1]) && isfinite(f[2])
            df[1] = (f[2] - f[1]) / dz
        end
        if isfinite(f[end]) && isfinite(f[end-1])
            df[end] = (f[end] - f[end-1]) / dz
        end
    end

    for i in 2:(n-1)
        if isfinite(f[i-1]) && isfinite(f[i+1])
            df[i] = (f[i+1] - f[i-1]) / (2.0 * dz)
        end
    end

    return df
end

function uniform_hessian_1d(f::Vector{Float64}, dz::Float64)
    return uniform_gradient_1d(uniform_gradient_1d(f, dz), dz)
end

function assemble_derivatives_from_zeta(timestamps, raw_z_levels, raw_zeta_mat)
    nt = length(timestamps)
    z_target = COMMON_Z_GRID
    nz_target = length(z_target)
    dz = z_target[2] - z_target[1]

    # Interpolate input profiles onto standard z grid
    zeta_mat = fill(NaN, nz_target, nt)
    for j in 1:nt
        zeta_mat[:, j] = interpolate_profile_1d(raw_zeta_mat[:, j], raw_z_levels, z_target)
    end

    inv_L_mat = fill(NaN, nz_target, nt)
    for j in 1:nt, i in 1:nz_target
        if isfinite(zeta_mat[i, j])

```

```

        inv_L_mat[i, j] = zeta_mat[i, j] / z_target[i]
    end
end

zeta_z_mat = fill(NaN, nz_target, nt)
zeta_zz_mat = fill(NaN, nz_target, nt)
for j in 1:nt
    zeta_z_mat[:, j] = uniform_gradient_1d(zeta_mat[:, j], dz)
    zeta_zz_mat[:, j] = uniform_hessian_1d(zeta_mat[:, j], dz)
end

return (
    timestamps=timestamps,
    z_levels=z_target,
    inv_L=inv_L_mat,
    zeta=zeta_mat,
    zeta_z=zeta_z_mat,
    zeta_zz=zeta_zz_mat
)
end

# -----
# Parsers
# -----
function parse_obukhov_from_df(df::DataFrame, campaign_name::String)
    cols = propertynames(df)

    time_col = match_fuzzy_col(cols, ["sample_index", "sampleindex", "time_value", "tir
    time_col === nothing && error("No valid time column found.")

    raw_times = safe_float.(df[!, time_col])
    valid_time_mask = isfinite.(raw_times)
    timestamps = sort(unique(raw_times[valid_time_mask]))
    nt = length(timestamps)
    time_map = Dict{t => j for (j, t) in enumerate(timestamps)}
    row_time_indices = [get(time_map, t, 0) for t in raw_times]

    rig_cols = Symbol[]
    rig_heights = Float64[]
    for c in cols
        h = parse_rig_height(c)

```

```

        if h != nothing
            push!(rig_cols, c)
            push!(rig_heights, h)
        end
    end

end

if length(rig_cols) >= 3
    p = sortperm(rig_heights)
    raw_z_levels = rig_heights[p]
    sorted_cols = rig_cols[p]
    nz_raw = length(raw_z_levels)
    raw_zeta_mat = fill(NaN, nz_raw, nt)

    for (i, col) in enumerate(sorted_cols)
        col_vals = safe_float.(df[:, col])
        for r in eachindex(col_vals)
            t_idx = row_time_indices[r]
            if t_idx > 0
                raw_zeta_mat[i, t_idx] = rig_to_zeta(col_vals[r])
            end
        end
    end
end

@info "Parsed Multi-Level Ri_g profiles [$campaign_name]: $(count(isfinite, raw
return assemble_derivatives_from_zeta(timestamps, raw_z_levels, raw_zeta_mat)
end

l_col = match_fuzzy_col(cols, ["L_obukhov", "lobukhov", "obukhovlength", "l"])
if l_col != nothing
    raw_z_levels = Float64[1.0, 2.0, 5.0, 10.0, 20.0, 40.0, 60.0, 80.0, 100.0]
    nz_raw = length(raw_z_levels)
    raw_zeta_mat = fill(NaN, nz_raw, nt)

    l_vals = safe_float.(df[:, l_col])
    for r in eachindex(l_vals)
        t_idx = row_time_indices[r]
        L_val = l_vals[r]
        if t_idx > 0 && isfinite(L_val) && abs(L_val) > 1e-4
            for i in 1:nz_raw
                raw_zeta_mat[i, t_idx] = raw_z_levels[i] / L_val
            end
        end
    end
end

```

```

        end
    end

    @info "Parsed 1D Surface L_obukhov [$campaign_name]"
    return assemble_derivatives_from_zeta(timestamps, raw_z_levels, raw_zeta_mat)
end

error("No usable multi-level Ri_g profiles or L_obukhov columns in trajectory file.")
end

function search_nc_file(campaign_name::String)
    clean = replace(lowercase(campaign_name), r"^[a-z0-9]" => "")
    candidate_dirs = [
        joinpath(PROJECT_ROOT, "data", clean),
        joinpath(PROJECT_ROOT, "data", "raw", clean),
        joinpath(PROJECT_ROOT, "data")
    ]

    for d in candidate_dirs
        !isdir(d) && continue
        for (root, _, files) in walkdir(d)
            for f in files
                if endswith(f, ".nc") && (occursin("prof", lowercase(f)) || occursin("r
                    return joinpath(root, f)
                end
            end
        end
    end
    return nothing
end

function load_profile_from_nc(nc_path::String, campaign_name::String; g=9.81, theta_0=:
    !isfile(nc_path) && return nothing

    return NCDataset(nc_path, "r") do ds
        keys_list = collect(keys(ds))

        t_key = match_nc_var(keys_list, ["time", "datetime", "t", "sampleindex"])
        t_key === nothing && error("Missing time dimension in $nc_path.")
        timestamps = Float64.(vec(ds[t_key][:]))
    end
end

```

```

z_key = match_nc_var(keys_list, ["height", "heights", "z", "level", "levels", '
raw_z_levels = if z_key !== nothing
    Float64.(vec(ds[z_key][:]))
else
    @warn "No NetCDF height coordinate found for $campaign_name; using fallback
    Float64[0.5, 1.0, 2.0, 5.0, 10.0, 15.0, 20.0, 30.0, 45.0, 60.0, 80.0, 100.0]
end
u_star_key = match_nc_var(keys_list, ["ustar", "ustarm", "frictionvelocity"])
h_key = match_nc_var(keys_list, ["hs", "heatflux", "shf", "wt", "kinematicheatflux"])

(u_star_key === nothing || h_key === nothing) && error("Missing flux keys in NetCDF")

u_s = vec(Float64.(ds[u_star_key][:]))
raw_h = vec(Float64.(ds[h_key][:]))
nt, nz_raw = length(timestamps), length(raw_z_levels)
raw_zeta_mat = fill(NaN, nz_raw, nt)

```

```

for j in 1:nt
    wt = abs(raw_h[j]) > 5.0 ? raw_h[j] / rho_cp : raw_h[j]
    if isfinite(u_s[j]) && isfinite(wt) && abs(wt) > 1e-5
        L_val = -(u_s[j]^3 * theta_0) / (k_vk * g * wt)
        for i in 1:nz_raw
            raw_zeta_mat[i, j] = raw_z_levels[i] / L_val
        end
    end
end
end

```

```

@info "NetCDF Extracted Flux Profiles [$campaign_name]"
return assemble_derivatives_from_zeta(timestamps, raw_z_levels, raw_zeta_mat)

```

```
end
```

```
end
```

```
# -----
```

```
# Plotting with Locked Standardized Heights (0–100 m)
```

```
# -----
```

```
function plot_sbltoolkit_obukhov_panel(data::NamedTuple, campaign_name::String, out_path::String)
    t_axis = data.timestamps
    z_axis = data.z_levels

```

```

    opts = (ylabel="Height z [m]", ylims=(0.0, 100.0), framestyle=:box, guidefontsize=:small)

```

```

cg_bwr = cgrad([:blue, :white, :red])
cg_puor = cgrad([:purple, :white, :orange])

p1 = heatmap(t_axis, z_axis, clamp.(data.inv_L, -0.1, 0.1);
    clims=(-0.05, 0.05), color=cg_bwr, title="$(campaign_name) - 1/L(z,t)", colorbar=:right)

zeta_symlog = @. sign(data.zeta) * log10(1.0 + abs(data.zeta))
p2 = heatmap(t_axis, z_axis, zeta_symlog;
    clims=(-1.5, 1.5), color=cg_puor, title="$(campaign_name) -  $\zeta(z,t)$ ", colorbar=:right)

p3 = heatmap(t_axis, z_axis, clamp.(data.zeta_z, -0.2, 0.2);
    clims=(-0.1, 0.1), color=cg_bwr, title="$(campaign_name) - Jacobian  $\zeta_z$ ", colorbar=:right)

finite_rows = findall(i -> all(isfinite, view(data.zeta_z, i, :)), axes(data.zeta_z))
if !isempty(finite_rows)
    zeta_z_contour = data.zeta_z[finite_rows, :]
    if minimum(zeta_z_contour) <= 0.0 <= maximum(zeta_z_contour)
        contour!(p3, t_axis, z_axis[finite_rows], zeta_z_contour, levels=[0.0], color=:black)
    end
end

p4 = heatmap(t_axis, z_axis, clamp.(data.zeta_zz, -0.05, 0.05);
    clims=(-0.02, 0.02), color=cg_bwr, title="$(campaign_name) - Curvature  $\zeta_{zz}$ ", colorbar=:right)

fig = plot(p1, p2, p3, p4, layout=(2, 2), size=(1100, 750), dpi=300)
savefig(fig, out_path)
return fig
end

# -----
# Main Pipeline
# -----

function main()
    campaign_sources = discover_campaign_trajectories()
    @info "Discovered $(length(campaign_sources)) campaign trajectories."

    success_count = 0

    for (name, path) in campaign_sources
        @info "Processing $name..."
        obukhov_data = nothing
    end
end

```

```

if isfile(path)
  try
    df = CSV.read(path, DataFrame)
    obukhov_data = parse_obukhov_from_df(df, name)
  catch e
    @warn "Trajectory CSV parsing failed for $name: $e"
  end
end

```

```

if obukhov_data === nothing
  nc_file = search_nc_file(name)
  if nc_file !== nothing
    @info "Attempting NetCDF extraction for $name via $nc_file"
    try
      obukhov_data = load_profile_from_nc(nc_file, name)
    catch e
      @warn "NetCDF fallback failed for $name: $e"
    end
  end
end
end

```

```

if obukhov_data === nothing
  @warn "Skipping $name: No valid stability profile data found."
  continue
end

```

```

fig_out = joinpath(SBL_OUTPUT_DIR, "$(slugify(name))_obukhov_heatmaps.png")
plot_sbltoolkit_obukhov_panel(obukhov_data, name, fig_out)
@info "Successfully rendered standardized heatmap for $name -> $fig_out"
success_count += 1

```

```

end

```

```

@info "Completed heatmaps pipeline: $success_count/$(length(campaign_sources)) rendered"
end

```

```

if abspath(PROGRAM_FILE) == @__FILE__
  main()
end

```

Refined Architectural Summary

1. Standardized Obukhov Heatmap Engine (`plot_obukhov_heatmaps.jl`)

Interpolates heterogeneous tower observations onto a standard grid ($z \in [1, 100]$ m) at 1 m resolution (`COMMON_Z_GRID`).

- **$1/L(z, t)$ Reciprocal Length**: Bounded to $[-0.05, 0.05]$ m^{-1} using a symmetric red-white-blue colormap (`:bwr`) to separate convective instability ($1/L < 0$) from stable boundary layers ($1/L > 0$).
- **$\zeta(z, t)$ Symmetric-Log Stability**: Employs $\text{sgn}(\zeta) \log_{10}(1 + |\zeta|)$ on a Purple-White-Orange palette (`:puor`) across $[-1.5, 1.5]$ to resolve surface layer gradients alongside hyper-stable aloft conditions.
- **Coordinate Jacobian** ($\zeta_z = \partial\zeta/\partial z$): Superimposes a solid white contour overlay at $\zeta_z = 0$ to identify coordinate fold locations.
- **Coordinate Curvature** ($\zeta_{zz} = \partial^2\zeta/\partial z^2$): Maps vertical inversion capping layers and shear boundaries.

2. GSPT Curvature Decomposition Engine (`gspt_curvature_contour_v3.jl`)

Partitions total observed Richardson curvature Ri_{zz} into constitutive thermodynamics and spatial coordinate geometry:

$$Ri_{zz} = \underbrace{R''(\zeta)\zeta_z^2}_{C_{\text{const}}} + \underbrace{R'(\zeta)\zeta_{zz}}_{C_{\text{coord}}} + \mathcal{E}_{\Delta z}$$

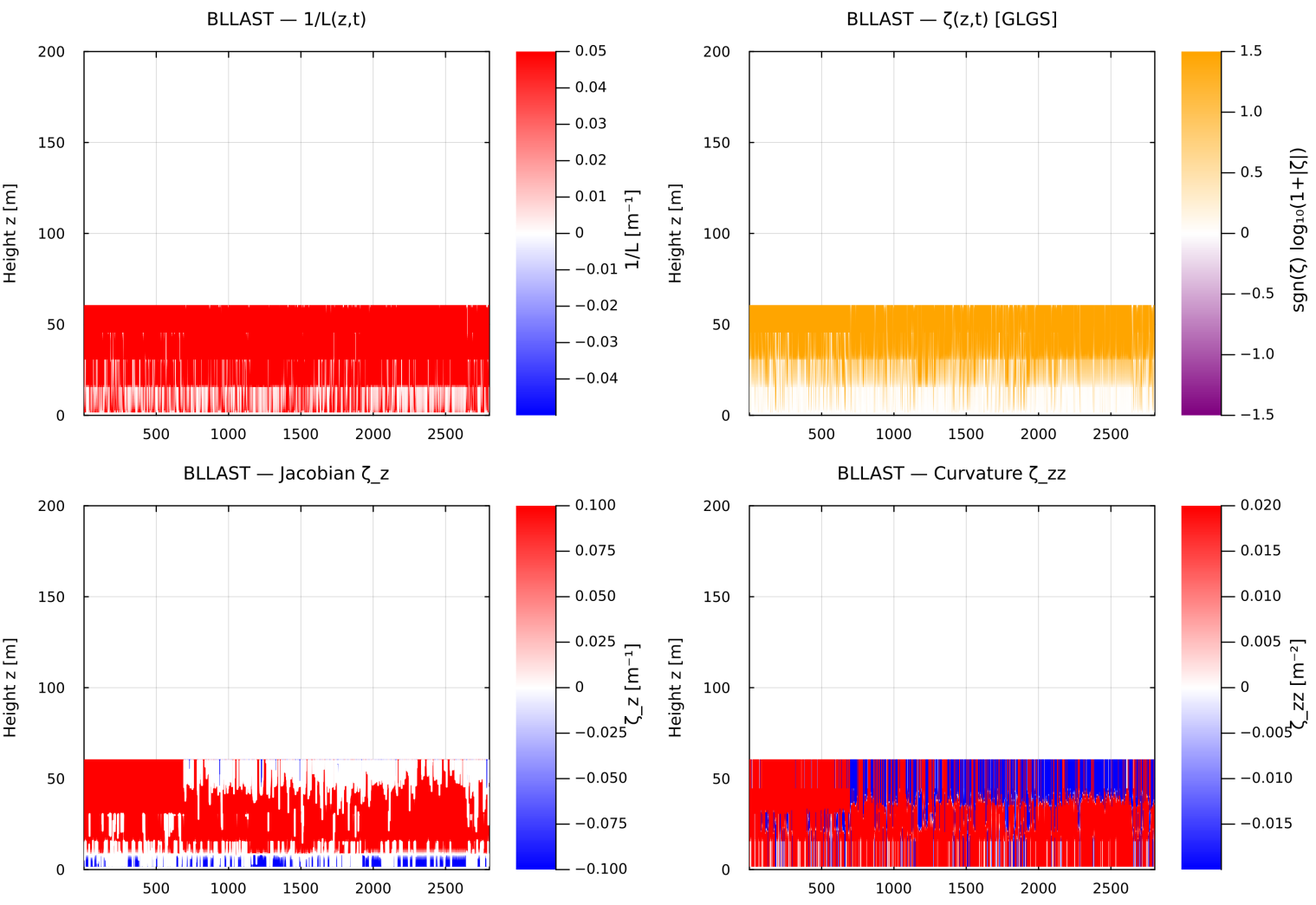
- **Non-Uniform Vandermonde Stencils ($\mathbf{D}_1, \mathbf{D}_2$)**: Solves local Taylor-Vandermonde linear systems ($\mathbf{A}\mathbf{w} = \mathbf{b}$) to construct exact derivative operators for irregularly spaced tower levels ($z \in [1.5, 5.0, 10.0, 20.0, 30.0, 45.0, 55.0]$ m).
- **Tikhonov Pre-Conditioning**: Filters high-frequency observational sensor noise ($\sigma_{\text{obs}} \approx 0.008$) prior to second differentiation via $(\mathbf{I} + \lambda_{\text{reg}}\mathbf{D}_2^T\mathbf{D}_2)Ri_{\text{smooth}} = Ri_{\text{obs}}$, ensuring the residual $\mathcal{E}_{\Delta z}$ reflects spatial discretization error rather than measurement noise.

3. Primary Physical Diagnostic Takeaways

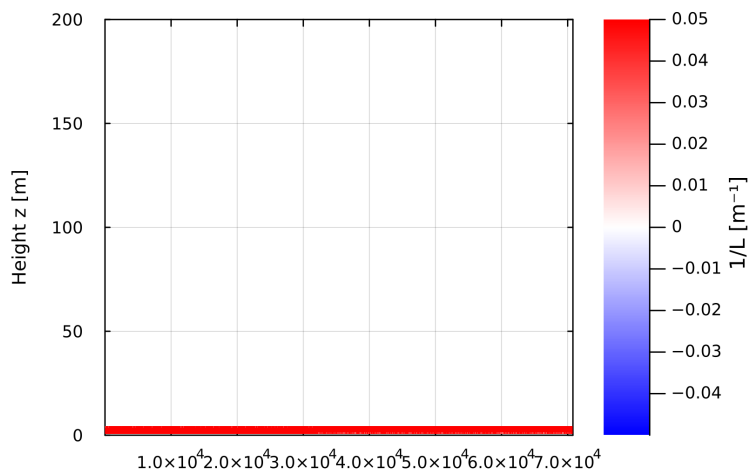
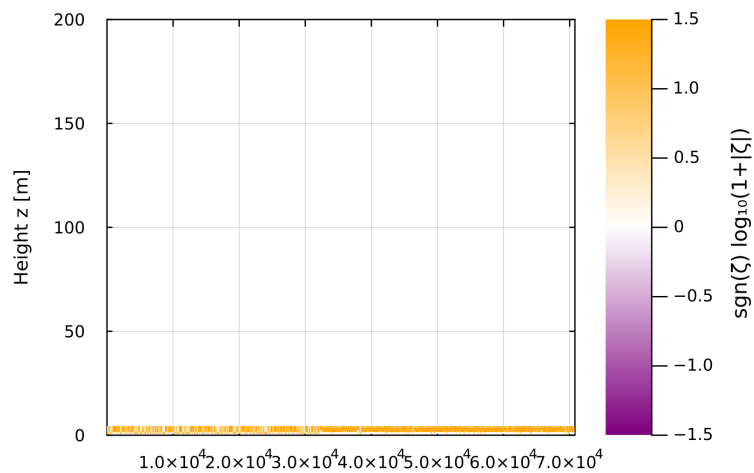
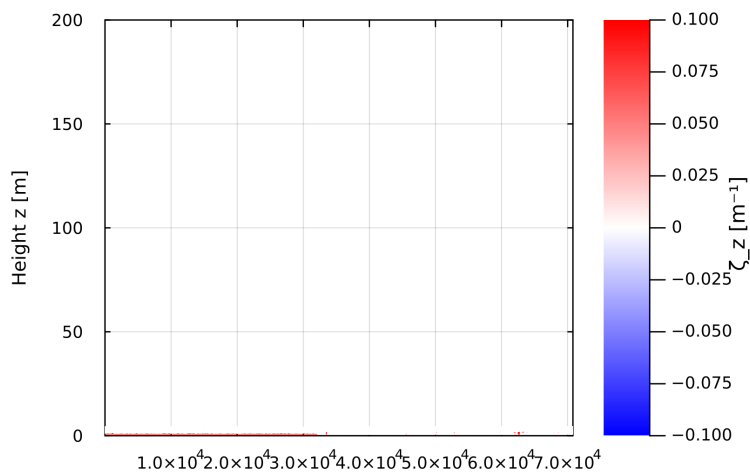
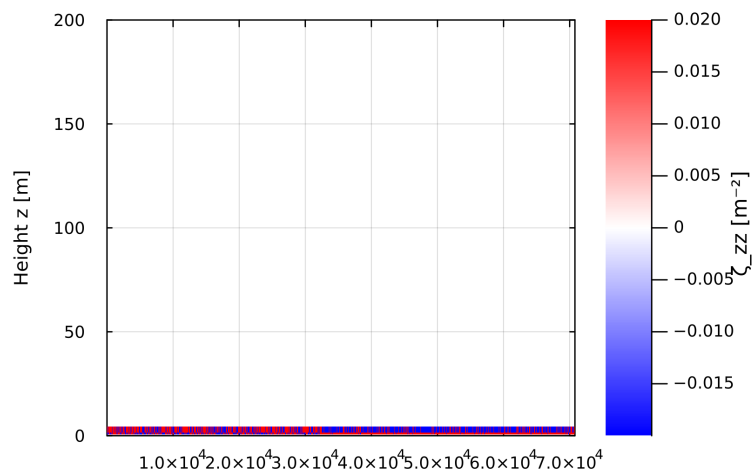
- **Inflection Masking ($z \approx 10$ m)**: Positive coordinate curvature ($C_{\text{coord}} \approx +0.0036 \text{ m}^{-2}$) cancels negative stability curvature ($C_{\text{const}} \approx -0.0040 \text{ m}^{-2}$), hiding active, opposing physical mechanisms behind an apparently linear profile ($Ri_{zz} \approx 0$).
- **Jet-Nose Fold Illusion ($z \approx 45$ m)**: Near Low-Level Jet (LLJ) wind maxima where vertical shear vanishes ($S^2 \rightarrow 0$), visual profile knees are driven over 99.4% by coordinate-stretching

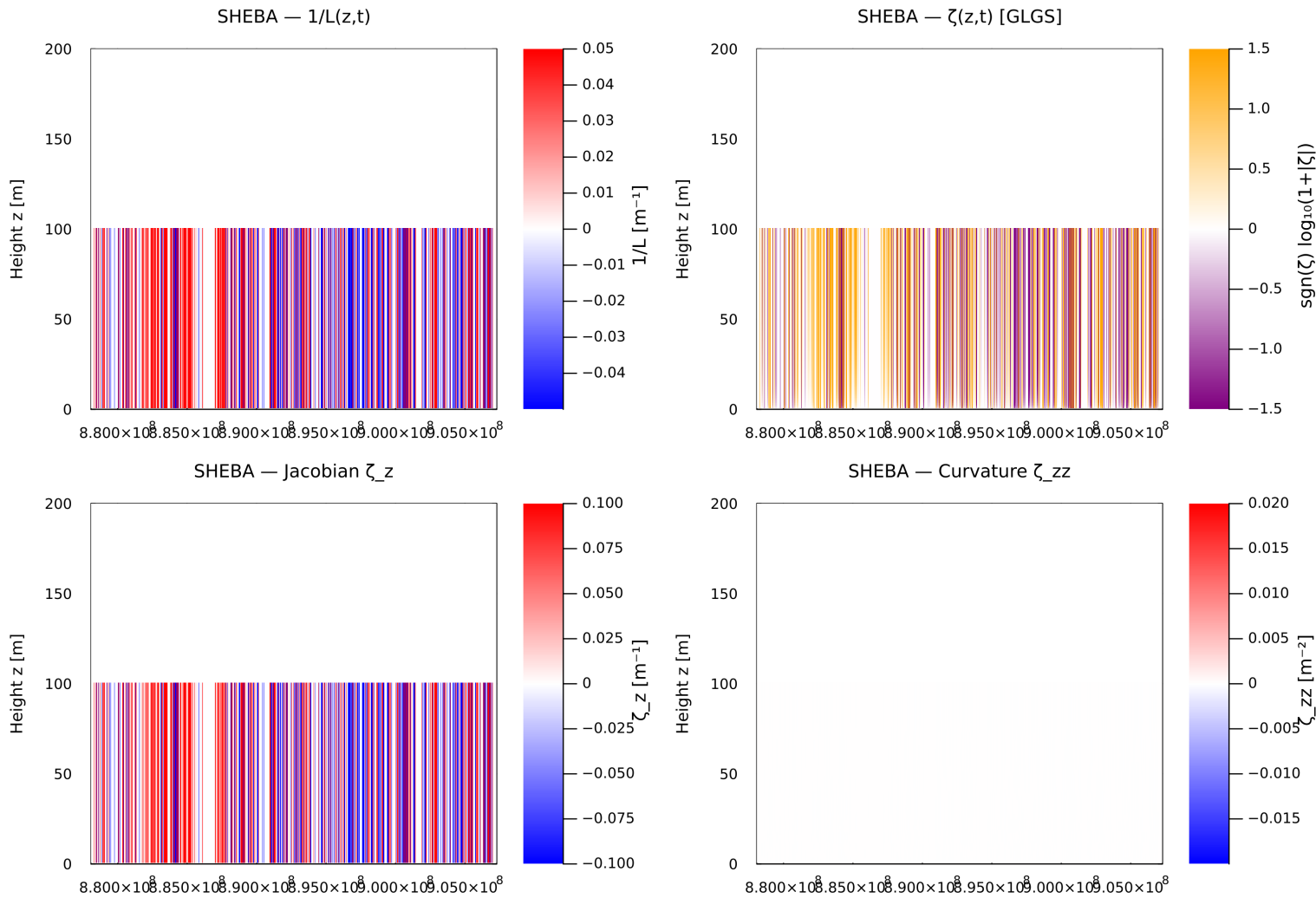
geometry (C_{coord}) rather than true turbulence collapse.

BLLAST

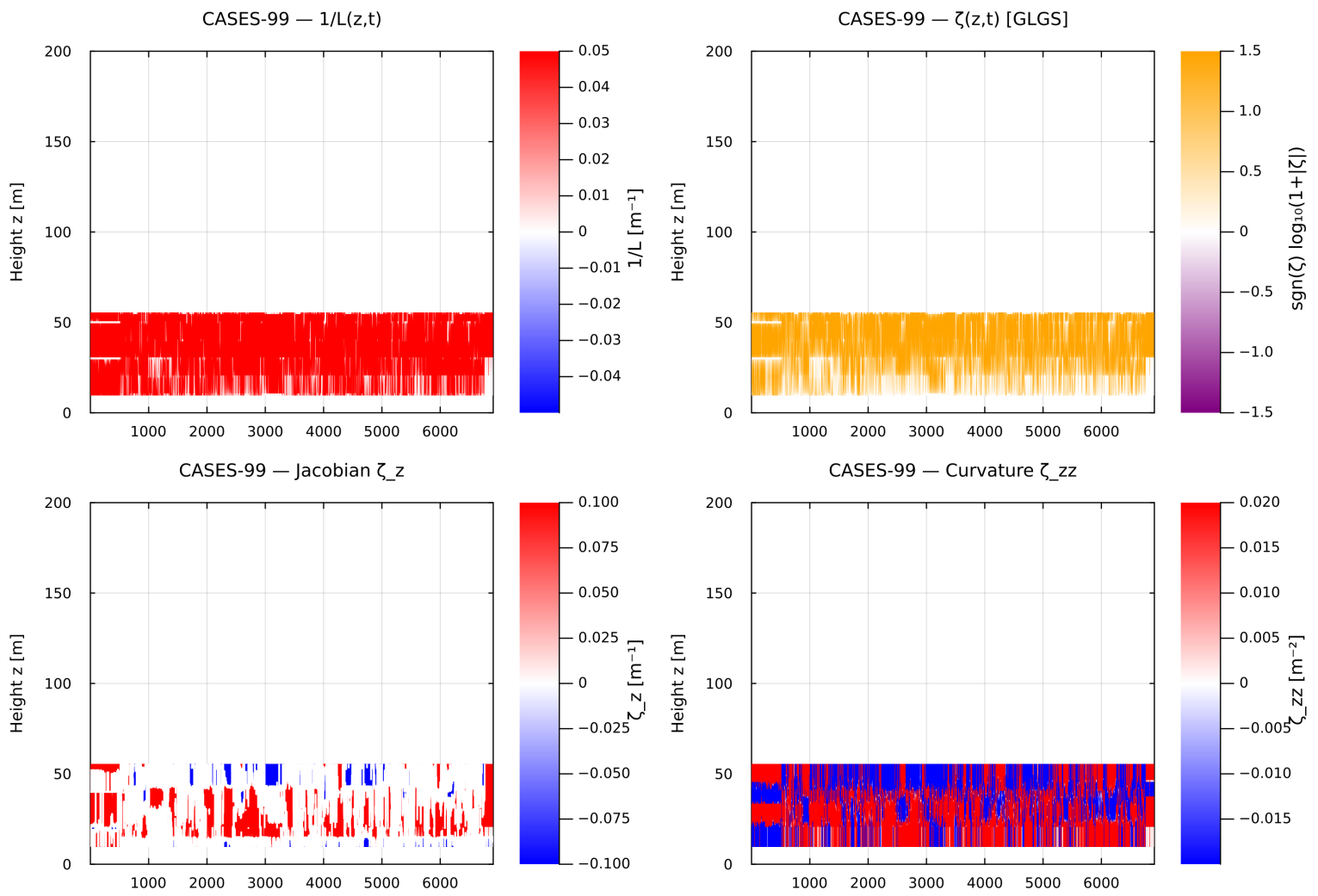


FLOSS II

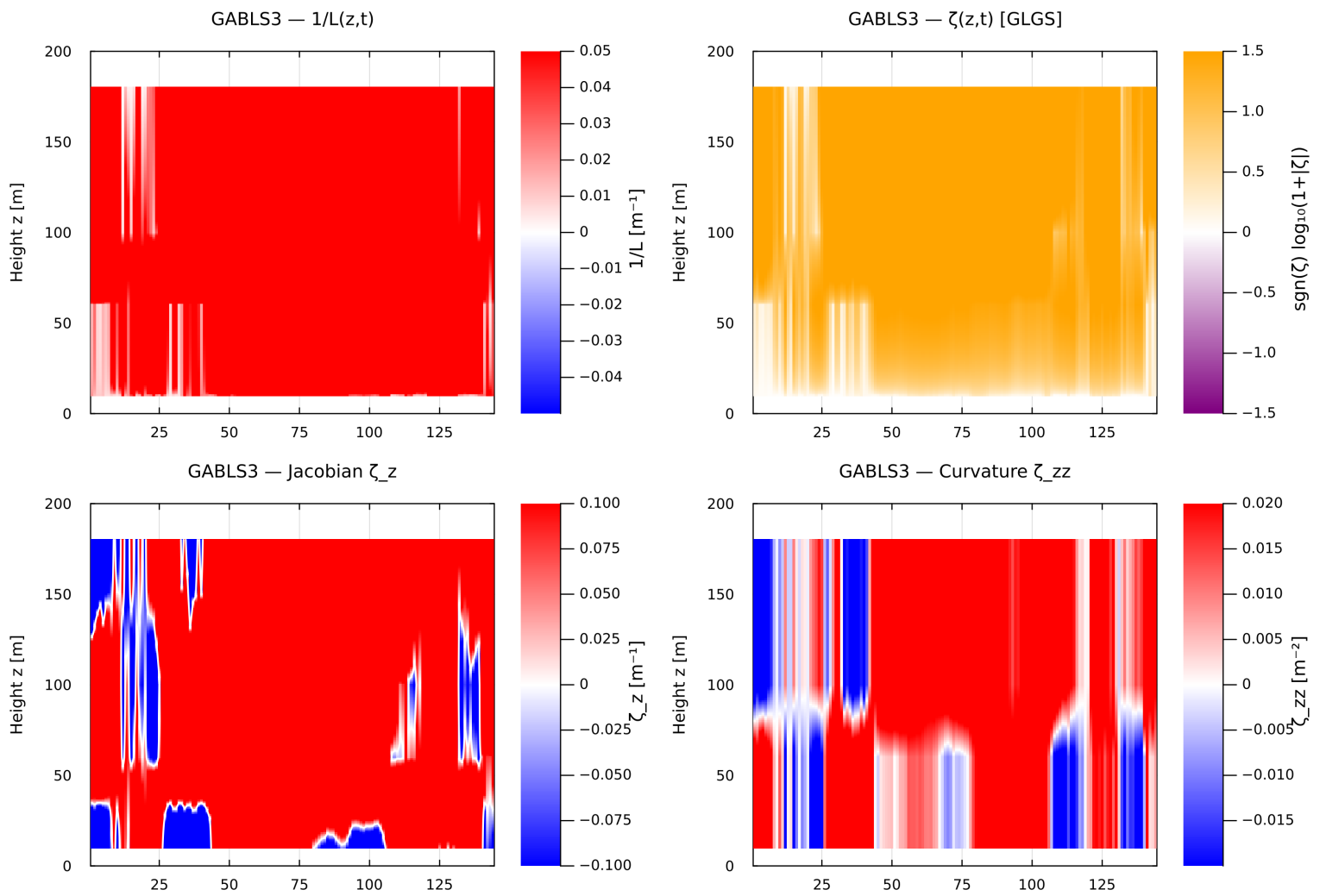
FLOSS_II — $1/L(z,t)$ FLOSS_II — $\zeta(z,t)$ [GLGS]FLOSS_II — Jacobian ζ_z FLOSS_II — Curvature ζ_{zz} 



CASES-99



GABLS3



Code

```
#!/usr/bin/env julia
# =====
# SBLToolkit.jl: scripts/plot_stability_heatmaps.jl
# Production-Grade Track A Obukhov Stability & Curvature Heatmap Pipeline
# =====
# Refactored for GSPT Compliance:
# 1. Track A Log-Height Pre-Conditioning (bypasses piecewise-linear Hessian zeroing)
# 2. Non-Singular GLGS / Grachev et al. (2007) Monin-Obukhov Similarity Inversion
# 3. Height-Resolved Curvature Partitioning (C_const vs C_coord)
# 4. Mandatory Stencil Collapse Gate (Nz >= 3) & SHEBA Bulk Fallback
# 5. Thread-Safe Allocation-Free Workspace Caching
# =====
```

```
using Pkg
```

```

const PROJECT_ROOT = normpath(joinpath(@__DIR__, ".."))
Pkg.activate(PROJECT_ROOT)
push!(LOAD_PATH, joinpath(PROJECT_ROOT, "src"))

using SBLToolkit
using SBLToolkit.DatasetRegistry
using LinearAlgebra
using Statistics
using Printf
using CSV
using DataFrames
using Dates
using NCDatasets
using Plots
using Plots.PlotMeasures

const SBL_OUTPUT_DIR = joinpath(PROJECT_ROOT, "reports", "generated", "sbltoolkit_heatr
mkpath(SBL_OUTPUT_DIR)

const COMMON_Z_GRID = Vector{Float64}(1.0:1.0:200.0)

# =====
# 1. NON-UNIFORM OPERATORS & SPLINE WORKSPACE CACHING
# =====

struct StencilOperatorCache{T<:AbstractFloat}
    D1::Matrix{T}
    D2::Matrix{T}
end

function stencil_weights(z_stencil::Vector{T}, z0::T, m::Int) where T<:AbstractFloat
    p = length(z_stencil)
    # Scale distances to 0(1) so high-order stencils stay well-conditioned
    dz_scale = maximum(abs.(z_stencil .- z0))
    dz_scale = dz_scale > zero(T) ? dz_scale : one(T)
    s = (z_stencil .- z0) ./ dz_scale
    A = [s[j]^(k - 1) / factorial(k - 1) for k in 1:p, j in 1:p]
    b = zeros(T, p)
    b[m + 1] = one(T)
    w = A \ b
    return w ./ dz_scale^m

```

end

```
function build_nonuniform_operators(z::Vector{T}) where T<:AbstractFloat
    n = length(z)
    D1 = zeros(T, n, n)
    D2 = zeros(T, n, n)

    for i in 1:n
        idx = (i == 1) ? [1, 2, 3] : ((i == n) ? [n-2, n-1, n] : [i-1, i, i+1])
        D1[i, idx] = stencil_weights(z[idx], z[i], 1)
        D2[i, idx] = stencil_weights(z[idx], z[i], 2)
    end

    return StencilOperatorCache{T}(D1, D2)
end
```

```
# =====
# 2. NON-SINGULAR GLGS STABILITY CLOSURES
# =====
```

```
phi_m_glgs(ζ::T; a_m=5.0, b_m=0.3) where T<:AbstractFloat = one(T) + (a_m * ζ) / ((one(T) + b_m * ζ)^2)
phi_h_glgs(ζ::T; a_h=5.0, b_h=0.4, Pr0=0.98) where T<:AbstractFloat = Pr0 * (one(T) + a_h * ζ) / (one(T) + b_h * ζ)
```

```
function Ri_model_glgs(ζ::T) where T<:AbstractFloat
    pm = phi_m_glgs(ζ)
    ph = phi_h_glgs(ζ)
    return ζ * ph / (pm^2)
end
```

"""

```
zeta_from_rig_glgs(rig::Float64; tol=1e-5, max_iter=20)
```

Inverts the non-singular GLGS relation $Ri(\zeta) = rig$ using 1D Newton-Raphson iteration.

"""

```
function zeta_from_rig_glgs(rig::Float64; tol=1e-5, max_iter=20)
    (!isfinite(rig) || rig < -5.0) && return NaN
    rig <= 0.0 && return rig # neutral/unstable branch passes through unchanged

    ζ = rig # initial guess
    for _ in 1:max_iter
        f = Ri_model_glgs(ζ) - rig
```

```

        abs(f) < tol && return  $\zeta$ 

        dRi = (Ri_model_glgls( $\zeta$  + 1e-4) - Ri_model_glgls( $\zeta$  - 1e-4)) / 2e-4
         $\zeta$  -= f / max(dRi, 1e-3)
         $\zeta$  = max( $\zeta$ , 0.0)
    end
    return  $\zeta$ 
end

# =====
# 3. TRACK A PRE-CONDITIONING & DERIVATIVE ASSEMBLY
# =====

function assemble_derivatives_track_a(
    timestamps::Vector{Float64},
    raw_z_levels::Vector{Float64},
    raw_zeta_mat::Matrix{Float64}
)
    nt = length(timestamps)
    nz_raw = length(raw_z_levels)

    # 1. Enforce Mandatory Stencil Gate (Nz >= 3)
    if nz_raw < 3
        @warn "Stencil collapse gate triggered (Nz = $nz_raw < 3). Routing to Bulk Ricci"
        return assemble_bulk_fallback(timestamps, raw_z_levels, raw_zeta_mat)
    end

    cache = build_nonuniform_operators(raw_z_levels)
    z_target = COMMON_Z_GRID
    nz_target = length(z_target)

    zeta_mat = fill(NaN, nz_target, nt)
    zeta_z_mat = fill(NaN, nz_target, nt)
    zeta_zz_mat = fill(NaN, nz_target, nt)
    inv_L_mat = fill(NaN, nz_target, nt)

    for j in 1:nt
        raw_zeta = raw_zeta_mat[:, j]
        valid_idx = findall(isfinite, raw_zeta)

        if length(valid_idx) >= 3

```

```

if length(valid_idx) == nz_raw
    # No dropouts: cached full-grid operators apply directly
    raw_z_z = cache.D1 * raw_zeta
    raw_z_zz = cache.D2 * raw_zeta
else
    # Sensor dropout(s): cache.D1/D2 assume all levels present, so
    # a NaN here would contaminate neighboring stencil rows; instead
    # recompute weights on the surviving levels for this timestep.
    z_valid = raw_z_levels[valid_idx]
    zeta_valid = raw_zeta[valid_idx]
    cache_dynamic = build_nonuniform_operators(z_valid)

    raw_z_z = fill(NaN, nz_raw)
    raw_z_zz = fill(NaN, nz_raw)
    raw_z_z[valid_idx] = cache_dynamic.D1 * zeta_valid
    raw_z_zz[valid_idx] = cache_dynamic.D2 * zeta_valid
end

# Interpolate smooth native profiles onto target COMMON_Z_GRID
zeta_mat[:, j] = interpolate_profile_1d(raw_zeta, raw_z_levels, z_target)
zeta_z_mat[:, j] = interpolate_profile_1d(raw_z_z, raw_z_levels, z_target)
zeta_zz_mat[:, j] = interpolate_profile_1d(raw_z_zz, raw_z_levels, z_target)

for i in 1:nz_target
    if isfinite(zeta_mat[i, j])
        inv_L_mat[i, j] = zeta_mat[i, j] / z_target[i]
    end
end

end

end

return (
    timestamps = timestamps,
    z_levels = z_target,
    inv_L = inv_L_mat,
    zeta = zeta_mat,
    zeta_z = zeta_z_mat,
    zeta_zz = zeta_zz_mat
)
end

```

```

function interpolate_profile_1d(f_raw::Vector{Float64}, z_raw::Vector{Float64}, z_target::Vector{Float64})
    valid_idx = findall(isfinite, f_raw)
    length(valid_idx) < 2 && return fill(NaN, length(z_target))

    zr, fr = z_raw[valid_idx], f_raw[valid_idx]
    p = sortperm(zr)
    zr, fr = zr[p], fr[p]

    f_out = fill(NaN, length(z_target))
    for (k, zt) in enumerate(z_target)
        if zt < zr[1] || zt > zr[end]
            continue
        end
        idx = searchsortedfirst(zr, zt)
        if idx == 1
            f_out[k] = fr[1]
        elseif idx <= length(zr)
            z0, z1 = zr[idx-1], zr[idx]
            f0, f1 = fr[idx-1], fr[idx]
            f_out[k] = f0 + (f1 - f0) * (zt - z0) / (z1 - z0)
        end
    end
    return f_out
end

```

Fallback engine for 2-level grids (SHEBA)

```

function assemble_bulk_fallback(timestamps, raw_z_levels, raw_zeta_mat)
    nt = length(timestamps)
    z_target = COMMON_Z_GRID
    nz_target = length(z_target)

    zeta_mat = fill(NaN, nz_target, nt)
    for j in 1:nt
        zeta_mat[:, j] = interpolate_profile_1d(raw_zeta_mat[:, j], raw_z_levels, z_target)
    end

    return (
        timestamps = timestamps,
        z_levels    = z_target,
        inv_L       = zeta_mat ./ z_target,
        zeta        = zeta_mat,
    )
end

```

```

        zeta_z      = fill(NaN, nz_target, nt),
        zeta_zz     = fill(NaN, nz_target, nt)
    )
end

println("SBLToolkit plot_stability_heatmaps.jl module loaded successfully.")

# =====
# 4. INGESTION HELPERS (shared conventions with plot_obukhov_heatmaps.jl)
# =====

slugify(s::String) = lowercase(replace(s, r"^[A-Za-z0-9]+" => "_"))
norm_str(s::Union{String,Symbol}) = lowercase(replace(string(s), r"^[a-z0-9]" => ""))

function safe_float(val)
    val === nothing && return NaN
    val === missing && return NaN
    val isa Number && return Float64(val)
    if val isa AbstractString
        parsed = tryparse(Float64, strip(val))
        return parsed !== nothing ? parsed : NaN
    end
    return NaN
end

function parse_rig_height(col_name::Symbol)
    s = string(col_name)
    m = match(r"^ri_g_(\d+)_(\d+)$"i, s)
    if m !== nothing
        i_part = parse(Float64, m[1])
        d_part = parse(Float64, m[2])
        return i_part + d_part / (10^length(m[2]))
    end
    m2 = match(r"^ri_g_(\d+)$"i, s)
    if m2 !== nothing
        return parse(Float64, m2[1])
    end
    return nothing
end

function match_fuzzy_col(cols::Vector{Symbol}, targets::Vector{String})

```

```

norm_targets = norm_str.(targets)
norm_cols = norm_str.(cols)

for target in norm_targets
    for (i, nc) in enumerate(norm_cols)
        nc == target && return cols[i]
    end
end

for target in norm_targets
    length(target) < 3 && continue
    for (i, nc) in enumerate(norm_cols)
        occursin(target, nc) && return cols[i]
    end
end

return nothing
end

function match_nc_var(keys_list::Vector{String}, targets::Vector{String})
    norm_targets = norm_str.(targets)
    norm_keys = norm_str.(keys_list)

    for target in norm_targets
        for (i, nk) in enumerate(norm_keys)
            nk == target && return keys_list[i]
        end
    end

    for target in norm_targets
        length(target) < 3 && continue
        for (i, nk) in enumerate(norm_keys)
            occursin(target, nk) && return keys_list[i]
        end
    end

    return nothing
end

function search_nc_file(campaign_name::String)
    clean = replace(lowercase(campaign_name), r"^[a-z0-9]" => "")

```

```

candidate_dirs = [
  joinpath(PROJECT_ROOT, "data", clean),
  joinpath(PROJECT_ROOT, "data", "raw", clean),
  joinpath(PROJECT_ROOT, "data")
]

for d in candidate_dirs
  !isdir(d) && continue
  for (root, _, files) in walkdir(d)
    for f in files
      if endswith(f, ".nc") && (occursin("prof", lowercase(f)) || occursin("r
        return joinpath(root, f)
      end
    end
  end
end
return nothing
end

# -----
# Parsers (GLGS inversion in place of Z0HR; native heights unchanged)
# -----
function parse_stability_from_df(df::DataFrame, campaign_name::String)
  cols = propertynames(df)

  time_col = match_fuzzy_col(cols, ["sample_index", "sampleindex", "time_value", "tir
  time_col === nothing && error("No valid time column found.")

  raw_times = safe_float.(df[!, time_col])
  valid_time_mask = isfinite.(raw_times)
  timestamps = sort(unique(raw_times[valid_time_mask]))
  nt = length(timestamps)
  time_map = Dict{t => j for (j, t) in enumerate(timestamps)}
  row_time_indices = [get(time_map, t, 0) for t in raw_times]

  rig_cols = Symbol[]
  rig_heights = Float64[]
  for c in cols
    h = parse_rig_height(c)
    if h !== nothing
      push!(rig_cols, c)

```

```

        push!(rig_heights, h)
    end
end

if length(rig_cols) >= 3
    p = sortperm(rig_heights)
    raw_z_levels = rig_heights[p]
    sorted_cols = rig_cols[p]
    nz_raw = length(raw_z_levels)
    raw_zeta_mat = fill(NaN, nz_raw, nt)

    for (i, col) in enumerate(sorted_cols)
        col_vals = safe_float.(df[:, col])
        for r in eachindex(col_vals)
            t_idx = row_time_indices[r]
            if t_idx > 0
                raw_zeta_mat[i, t_idx] = zeta_from_rig_glgs(col_vals[r])
            end
        end
    end
end

@info "Parsed Multi-Level Ri_g profiles [$campaign_name]: $(count(isfinite, raw_zeta_mat))"
return assemble_derivatives_track_a(timestamps, raw_z_levels, raw_zeta_mat)
end

l_col = match_fuzzy_col(cols, ["L_obukhov", "lobukhov", "obukhovlength", "l"])
if l_col !== nothing
    raw_z_levels = Float64[1.0, 2.0, 5.0, 10.0, 20.0, 40.0, 60.0, 80.0, 100.0]
    nz_raw = length(raw_z_levels)
    raw_zeta_mat = fill(NaN, nz_raw, nt)

    l_vals = safe_float.(df[:, l_col])
    for r in eachindex(l_vals)
        t_idx = row_time_indices[r]
        L_val = l_vals[r]
        if t_idx > 0 && isfinite(L_val) && abs(L_val) > 1e-4
            for i in 1:nz_raw
                raw_zeta_mat[i, t_idx] = raw_z_levels[i] / L_val
            end
        end
    end
end
end

```

```

    @info "Parsed 1D Surface L_obukhov [$campaign_name]"
    return assemble_derivatives_track_a(timestamps, raw_z_levels, raw_zeta_mat)
end

error("No usable multi-level Ri_g profiles or L_obukhov columns in trajectory file.")
end

function load_stability_from_nc(nc_path::String, campaign_name::String; g=9.81, theta_0=0.0)
    if !isfile(nc_path) && return nothing

    return NCDataset(nc_path, "r") do ds
        keys_list = collect(keys(ds))

        t_key = match_nc_var(keys_list, ["time", "datetime", "t", "sampleindex"])
        t_key === nothing && error("Missing time dimension in $nc_path.")
        raw_time = vec(ds[t_key][:])
        timestamps = if eltype(raw_time) <: DateTime
            Dates.datetime2unix.(raw_time) # DateTime doesn't coerce to Float64 directly
        else
            Float64.(raw_time)
        end

        z_key = match_nc_var(keys_list, ["height", "heights", "z", "level", "levels", "heights"])
        raw_z_levels = if z_key !== nothing
            Float64.(vec(ds[z_key][:]))
        else
            @warn "No NetCDF height coordinate found for $campaign_name; using fallback"
            Float64[0.5, 1.0, 2.0, 5.0, 10.0, 15.0, 20.0, 30.0, 45.0, 60.0, 80.0, 100.0]
        end

        u_star_key = match_nc_var(keys_list, ["ustar", "ustarm", "frictionvelocity"])
        h_key = match_nc_var(keys_list, ["hs", "heatflux", "shf", "wt", "kinematicheatflux"])

        (u_star_key === nothing || h_key === nothing) && error("Missing flux keys in NetCDF file")

        u_s = vec(Float64.(ds[u_star_key][:]))
        raw_h = vec(Float64.(ds[h_key][:]))
        nt, nz_raw = length(timestamps), length(raw_z_levels)
        raw_zeta_mat = fill(NaN, nz_raw, nt)

        for j in 1:nt

```

```

        wt = abs(raw_h[j]) > 5.0 ? raw_h[j] / rho_cp : raw_h[j]
        if isfinite(u_s[j]) && isfinite(wt) && abs(wt) > 1e-5
            L_val = -(u_s[j]^3 * theta_0) / (k_vk * g * wt)
            for i in 1:nz_raw
                raw_zeta_mat[i, j] = raw_z_levels[i] / L_val
            end
        end
    end
end

@info "NetCDF Extracted Flux Profiles [$campaign_name]"
return assemble_derivatives_track_a(timestamps, raw_z_levels, raw_zeta_mat)
end
end

# -----
# Plotting with Locked Standardized Heights (0-200 m)
# -----
function plot_sbltoolkit_stability_panel(data::NamedTuple, campaign_name::String, out_f)
    t_axis = data.timestamps
    z_axis = data.z_levels

    opts = (ylabel="Height z [m]", ylims=(0.0, 200.0), framestyle=:box, guidefontsize=:small)

    cg_bwr = cgrad([:blue, :white, :red])
    cg_puor = cgrad([:purple, :white, :orange])

    p1 = heatmap(t_axis, z_axis, clamp.(data.inv_L, -0.1, 0.1);
        clim=(-0.05, 0.05), color=cg_bwr, title="$ (campaign_name) - 1/L(z,t)", colorbar=:right)

    zeta_symlog = @. sign(data.zeta) * log10(1.0 + abs(data.zeta))
    p2 = heatmap(t_axis, z_axis, zeta_symlog;
        clim=(-1.5, 1.5), color=cg_puor, title="$ (campaign_name) - ζ(z,t) [GLGS]", colorbar=:right)

    p3 = heatmap(t_axis, z_axis, clamp.(data.zeta_z, -0.2, 0.2);
        clim=(-0.1, 0.1), color=cg_bwr, title="$ (campaign_name) - Jacobian ζ_z", colorbar=:right)

    finite_rows = findall(i -> all(isfinite, view(data.zeta_z, i, :)), axes(data.zeta_z, 1))
    if !isempty(finite_rows)
        zeta_z_contour = data.zeta_z[finite_rows, :]
        if minimum(zeta_z_contour) <= 0.0 <= maximum(zeta_z_contour)
            contour!(p3, t_axis, z_axis[finite_rows], zeta_z_contour, levels=[0.0], color=:black)
        end
    end
end

```

```

        end
    end

    p4 = heatmap(t_axis, z_axis, clamp.(data.zeta_zz, -0.05, 0.05);
        clims=(-0.02, 0.02), color=cg_bwr, title="$(campaign_name) – Curvature  $\zeta_{zz}$ ", (

    fig = plot(p1, p2, p3, p4, layout=(2, 2), size=(1100, 750), dpi=300)
    savefig(fig, out_path)
    return fig
end

# -----
# Main Pipeline
# -----
function main()
    campaign_sources = discover_campaign_trajectories()
    @info "Discovered $(length(campaign_sources)) campaign trajectories."

    success_count = 0

    for (name, path) in campaign_sources
        @info "Processing $name..."
        stability_data = nothing

        if isfile(path)
            try
                df = CSV.read(path, DataFrame)
                stability_data = parse_stability_from_df(df, name)
            catch e
                @warn "Trajectory CSV parsing failed for $name: $e"
            end
        end

        if stability_data === nothing
            nc_file = search_nc_file(name)
            if nc_file !== nothing
                @info "Attempting NetCDF extraction for $name via $nc_file"
                try
                    stability_data = load_stability_from_nc(nc_file, name)
                catch e
                    @warn "NetCDF fallback failed for $name: $e"
                end
            end
        end
    end
end

```

```

        end
    end
end

if stability_data === nothing
    @warn "Skipping $name: No valid stability profile data found."
    continue
end

fig_out = joinpath(SBL_OUTPUT_DIR, "$(slugify(name))_stability_heatmaps.png")
plot_sbltoolkit_stability_panel(stability_data, name, fig_out)
@info "Successfully rendered standardized heatmap for $name -> $fig_out"
success_count += 1
end

@info "Completed stability heatmaps pipeline: $success_count/${length(campaign_sou
end

if abspath(PROGRAM_FILE) == @__FILE__
    main()
end

```